OWASP Board Activity and Candidate Verification Framework

GSOC'26 Proposal for OWASP Foundation

About Me

I am a pre-final year Computer Science engineering student at MITS, Gwalior, India. I joined the OWASP Nest community in June last year and have been an active contributor to OWASP Nest since then. In this time, I have contributed **61+ merged commits** and have helped review 15+ Pull Requests. This experience has provided me with a deep understanding of the codebase.

Information about me:

- Email: rudransh.shr@gmail.com / rudransh.shrivastava@owasp.org
- GitHub: https://github.com/rudransh-shrivastava
- Twitter/X: https://x.com/rudranshstwt
- LinkedIn: https://linkedin.com/in/rudransh-shrivastava
- Slack Profile (OWASP Workspace): https://owasp.slack.com/team/U091EAFGDEG
- Timezone: Indian Standard Time (UTC+5:30)
- University: Pre-Final Year Engineering Student at Madhav Institute of Technology and Science, Gwalior

Previous Contributions to OWASP Nest

All my merged Pull Requests can be found using this link.

I led the recent infrastructure migration which migrates our single instance server to AWS serverless architecture:

PR Link	Title
#4292	Migrate Production
#4258, #4243, #4231, #4219	Preparation for Production Migration
#4187	Migrate Backend to ECS
#3996	Automate IAM User Policies

#3466	Add CI/CD for Staging Deployment
#3381	Use Lambda SnapStart and optimize Zappa package
#3694, #3691, #3681, #3679, #3627, #3581, #3567, #3499, #3333	Add tests and test CI for all Terraform modules
#3238	Optimize Costs for Staging Deployment
#2812, #2810, #2777, #2770, #2760	Improvements to Infrastructure code (database, ecs, networking, S3, pre-commit hooks, and quality improvements)
#2743	Add logging in Infrastructure
#2699	Add Remote State Management
#2685	Improve Security Groups for Infrastructure
#2551	Integrate AWS Parameter Store for Zappa/Infrastructure deployments
#2431	Migrate OWASP Nest to Zappa for serverless deployment

I am currently contributing to the NestBot development:

Link	**Title**
#4324 (in progress)	Implement Collaborative Flow for Complex Queries
#4303 (under review)	Implement Query Analyzer for NestBot
#3925	Add Image Extraction Support to NestBot

My team won the OWASP Nest API Hackathon which I followed up by integrating the hackathon project in the codebase in #2948 - Add Community Snapshot Videos Command

Contributions to the nest-schema repository: #65, #275, #276

Project maintenance contributions (CVE mitigations, bug-fixes, dependency version migrations, and more): #4142, #4133, #3861, #2528, #2416, #2362, #2331, #2223, #2178

Motivation and Expectations

As an OWASP membership holder, I voted for the 2025 Board of Directors elections. While reading the candidates' statements here, I found it difficult to verify the information. As a contributor to Nest, I knew the project had a dashboard to solve this problem. The dashboard included activity metrics from GitHub and Slack and had received suggestions/improvement ideas (source) from candidates.

This is a problem every voter faces, and I want to solve it by making candidate information more transparent and verifiable. I also want to bring structure and a standard to board activity data so it is programmatically accessible and auditable by the community.

Apart from growth as a developer/contributor, I expect to learn from the mentors at decision points where I cannot navigate alone.

Project Details

This project combines two ideas:

1. OWASP Board Candidate Information Transparency and Fact-Checking
2. OWASP Board Activity Standardization and Data Programmatic Access

Size: Large (350 Hours)
Mentors: Arkadii Yakovets, Kate Golovanova, TBD

Board Candidate Information Transparency and Fact-Checking

Project Abstract and Goals

The Board Candidate Information Transparency and Fact-Checking project aims to provide community members with up-to-date, verified, and clear candidate information to support informed decision making.
To achieve this, the existing Board of Director candidates dashboard will be improved to support additional verified candidate information. Two new dashboards will be introduced: one for board candidates to submit claims and track their review status, and one for OWASP staff to review submitted claims. The review process will be a collaborative effort, with claims requiring a minimum number of reviews before being accepted or rejected.

1. Implement a candidate claims submission dashboard enabling board candidates to submit claims and track their review status.

2. Implement an OWASP staff review dashboard enabling staff and community members to review, annotate, and fact-check candidate claims with full traceability of reviewer identity and timestamp.
3. Implement an AI powered parser to split candidate pages into actionable claims for human review.
4. Backfill 2025 BoD candidate data.
5. Implement a structured, multi-reviewer fact checking workflow that involves both community members and OWASP staff members.
6. Display approved candidate claims on candidate profiles in the existing dashboard including claim reviewers and supporting data.
7. Improve the existing BoD candidates dashboard to display candidate activity metrics like contributions/comments across PRs, issues, and discussions, add filter and sort options for verified data.

Related Work and Current State

The existing BoD candidates dashboard at `/board/<year>/candidates` showcases the Board of Directors candidates for a particular year.

Each candidate's profile displays activity data from OWASP GitHub organization contributions and the Slack workspace engagement data, which limits candidates to only demonstrating participation and contribution activity within these platforms.

There is no functionality for the candidate to add supporting information to their profiles. This information could include verifiable credentials, certifications, external leadership roles, conference participation, funding contributions, and campaign commitments. All of these are relevant to a candidate's suitability for the board but are currently invisible to community members reviewing the dashboard.

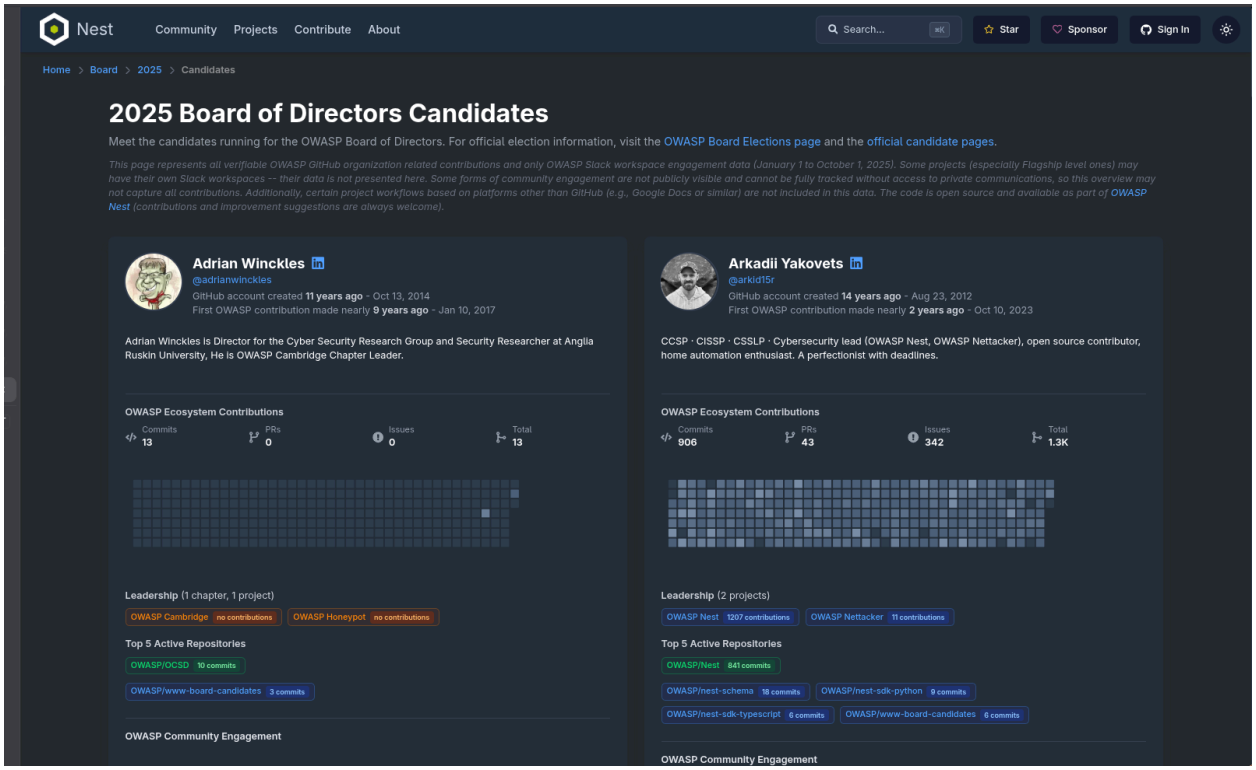

Figure 1 *Screenshot of current BoD Candidates dashboard for year 2025*

Benefits to the Community

Community members currently have no way to verify candidate claims beyond what is publicly visible on GitHub and Slack. This project addresses that directly.

Voters gain access to clear, verified information about candidates which supports informed decision making.

Candidates benefit from a formal platform to present their credentials, leadership roles, and commitments that are otherwise invisible in the existing dashboard.

A transparent review process gives community members confidence over their votes and is auditable at any point.

Approach and Implementation Plan

1. Candidate Claim Management

In this phase, I will focus on the frontend and backend implementation for managing candidate claims.

Some important points to keep in mind:

- A candidate claim may include supporting documents like `.pdf` files or links.
- A claim will have a status (`DRAFT`, `DISCARDED`, `SUBMITTED`, `APPROVED`, `REJECTED`, `WITHDRAWN`)
- A claim will become immutable if it gets finalized (`APPROVED` or `REJECTED`) or gets `WITHDRAWN`.
- A claim may only be submitted by the candidate.
- A claim may be withdrawn even after it's locked to respect the candidate's privacy.
- A claim may only be `DISCARDED` if it's in the `DRAFT` stage.

While keeping this in mind, I will create two Django models:

1. `BoardCandidateClaim`:
   - `id`: Unique PK
   - `board`: FK -> `BoardOfDirectors`
   - `candidate_id`: FK -> `EntityMember`
   - `description`: Text
   - `is_locked`: Boolean (default false, true once finalized or withdrawn)
   - `status`: Enum(`DRAFT`, `DISCARDED`, `SUBMITTED`, `APPROVED`, `REJECTED`, `WITHDRAWN`) (default `DRAFT`)
   - `title`: Text
   - `withdrawn_at`: Nullable Timestamp
   - `withdrawn_reason`: Nullable Text
   - Timestamps (`created_at`, `updated_at`) using `TimestampedModel` model.

Constraints:

- `is_locked` prevents any further updates and reviews to the candidate claim (Claim withdrawal is allowed).

2. `BoardCandidateClaimEvidence`:
   - `id`: Unique PK
   - `claim_id`: FK -> `BoardCandidateClaim`
   - `description`: Text
   - `filename`: Nullable text
   - `file_url`: Nullable text (URL for files uploaded by candidate)
   - `is_removed`: Boolean (default false)
   - `mime_type`: Nullable Text
   - `removed_at`: Nullable Timestamp
   - `removed_reason`: Nullable Text
   - `s3_key`: Nullable Text
   - `size`: Number
   - `source_url`: Nullable Text (external link, helpful if candidate attaches a file)

- ➢ `title`: Text
- ➢ Timestamps (created_at, updated_at) using `TimestampedModel` model.

Constraints:

- ➢ Either `file_url` or `s3_key` is required (use `clean()` method)
- ➢ Cannot be added to a locked claim.

Storage Options: File storage for local development and S3 buckets for staging and production (I will provision S3 buckets using Terraform).

I will implement a GraphQL query to expose this claim data with restrictions on who can view it (candidate only for now, Candidate + OWASP Staff + community volunteers in phase 2, no restrictions on `APPROVED` claims), and mutations to add/update claims (Only candidates can add/update their claims).

Then, I will design a dashboard `/board/<year>/candidates/<candidate-id>/claims` for the candidate to add/edit `DRAFT` claims and view the status of their `SUBMITTED` claims. This page will be reachable by an edit button in candidate profiles only the candidate can see. This page will also include a form to submit a claim, reachable by a "+ Submit Claim" button. A prototype for the UI for the dashboard is shown in Figure 2.

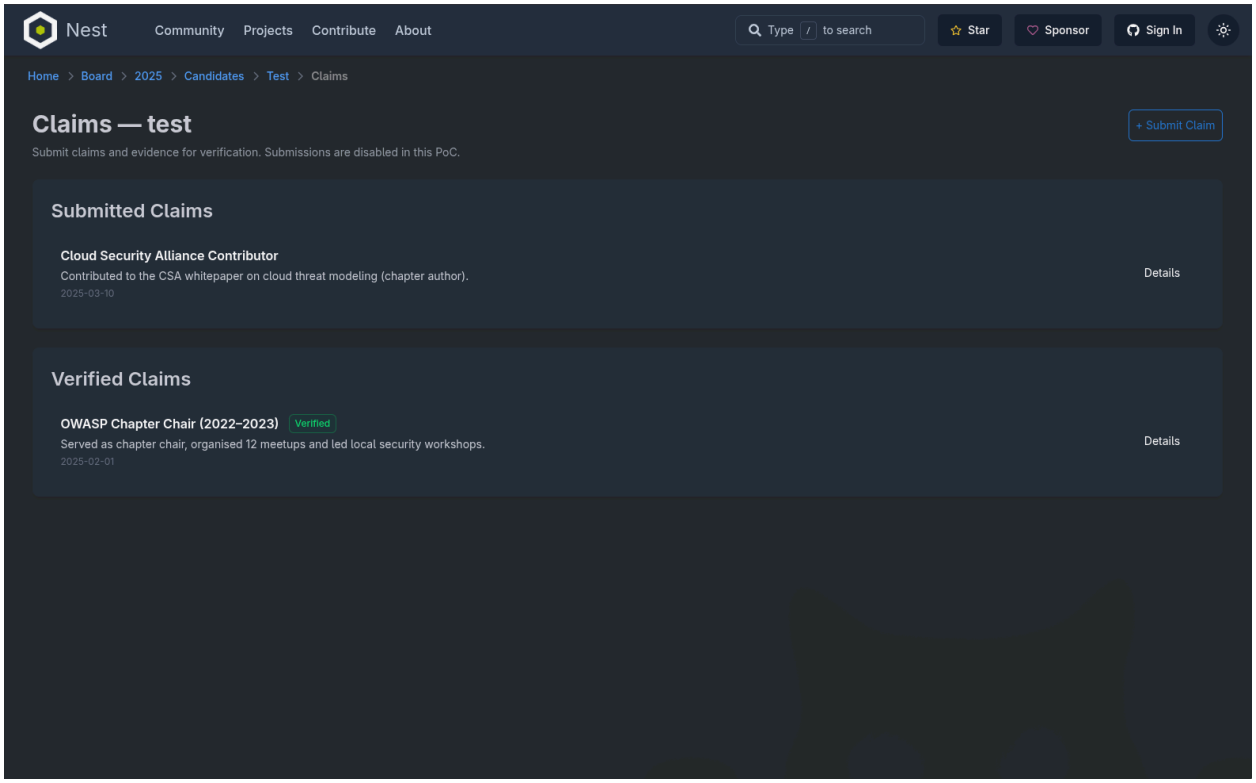

Figure 2 *Prototype design for the claims dashboard*

2. Claim Review Management

This phase focuses on adding the claim review feature. To achieve this, I will add a Django model to store the reviews, expose it via GraphQL mutations and design a frontend dashboard to submit and view reviews.

Some important points to keep in mind:

- ➢ Only OWASP Staff and community volunteers are authorized to leave reviews.
- ➢ A review is immutable once submitted.
- ➢ A reviewer can only submit one review per claim.
- ➢ A review cannot be added to a locked claim.
- ➢ A minimum number of positive reviews will be required for the claim to be finalized (configurable threshold)

While keeping this in mind, I will create a Django model:

`BoardCandidateClaimReviews`:

- ➢ `id`: Unique PK
- ➢ `claim_id`: FK -> `BoardCandidateClaim`
- ➢ `decision`: Enum(`APPROVED`, `REJECTED`)
- ➢ `notes`: Nullable Text
- ➢ `reviewer_id`: FK -> `EntityMember`
- ➢ Timestamp (created_at)

Constraints:

- ➢ Unique constraint (`claim_id`, `reviewer_id`)
- ➢ Community volunteers who are also candidates cannot review claims.

I will update the `BoardOfDirectors` model to include a configurable threshold for the minimum number of reviews required for a claim to be finalized. A `post_save` logic will be added to re-evaluate unverified claims when the threshold is changed (does not affect locked claims).

I will create a GraphQL mutation `createBoardCandidateClaimReview` used to create a candidate claim review with necessary authorization.

I will also design a dashboard at `/board/<year>/candidates/review` that only the OWASP Staff and community volunteers can access. This dashboard will allow them to submit their reviews for all candidate claims. A prototype for the design is shown in Figure 3.

Role assignment for the claims review system will extend the existing GitHub User model, which already carries an `is_owasp_staff` boolean field. A new `is_community_reviewer` boolean field will be added.

Authorization for `createBoardCandidateClaimReview` will require either `is_owasp_staff` or `is_community_reviewer` to be true. Candidates will be identified via the existing `EntityMember` relationship, and a user who is both a candidate and a community reviewer will be blocked from reviewing claims in the same election year.

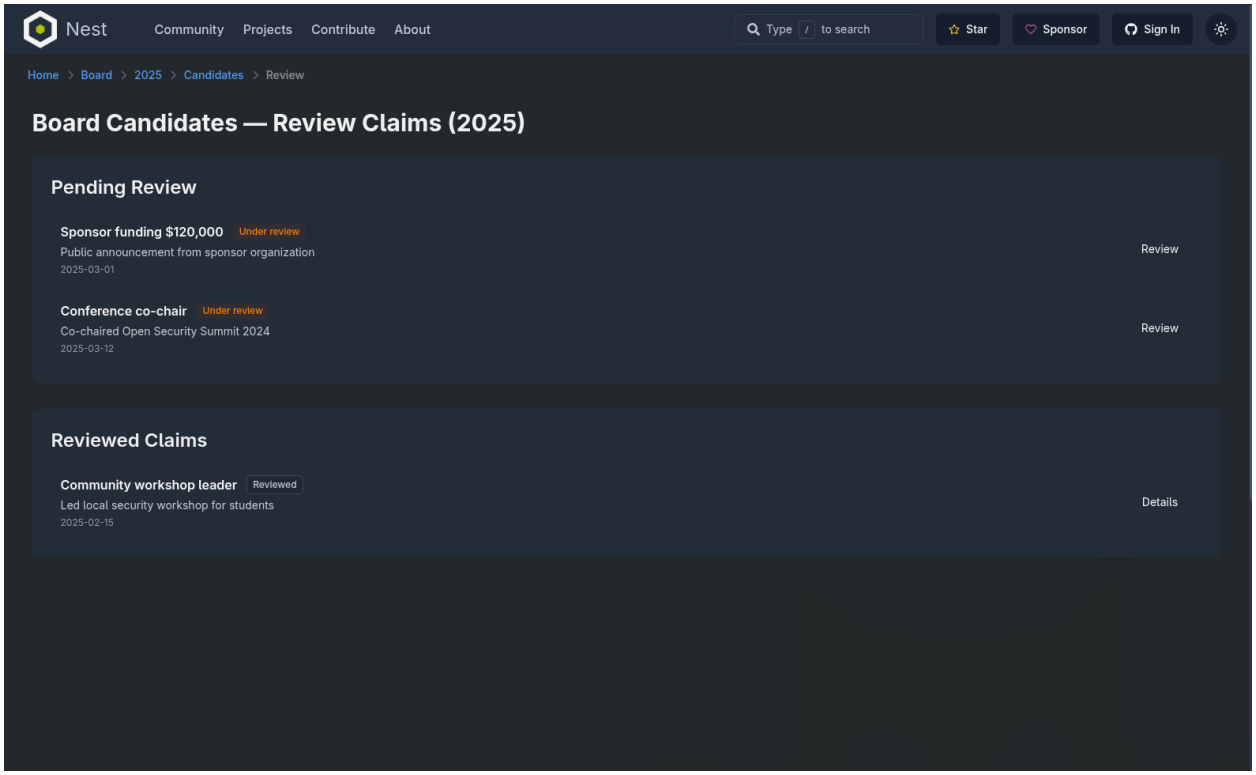

Figure 3 *Prototype design for claims review dashboard*

3. AI-Powered Claim Parsing

In this phase, I will add an AI-powered pipeline that fetches candidate statements from the www-board-candidates GitHub repository and splits them into smaller actionable draft claims using external LLM APIs.

To achieve this, I will add a Django management command that:

1. Fetches the candidate markdown file from `www-board-candidates` repository for a given election year using GitHub API.
2. Passes the markdown content to an external LLM API to extract verifiable claims.
3. Stores the extracted claims as `DRAFT BoardCandidateClaim` records linked to candidate and election year.

This command will be used for new candidates (candidates who have no prior record in our database). This will be helpful to backfill 2025 claims to be used for 2026 BoD elections.

4. Board of Directors Dashboard Improvements

In this phase, I will focus on improving the BoD dashboard by adding the verified claims that were added earlier. I will add additional activity metrics like comments on PRs, discussions, and issues to further improve the dashboard. Sort and Filter options to prioritize verified data will also be added.

A prototype for the design is shown in Figure 4.

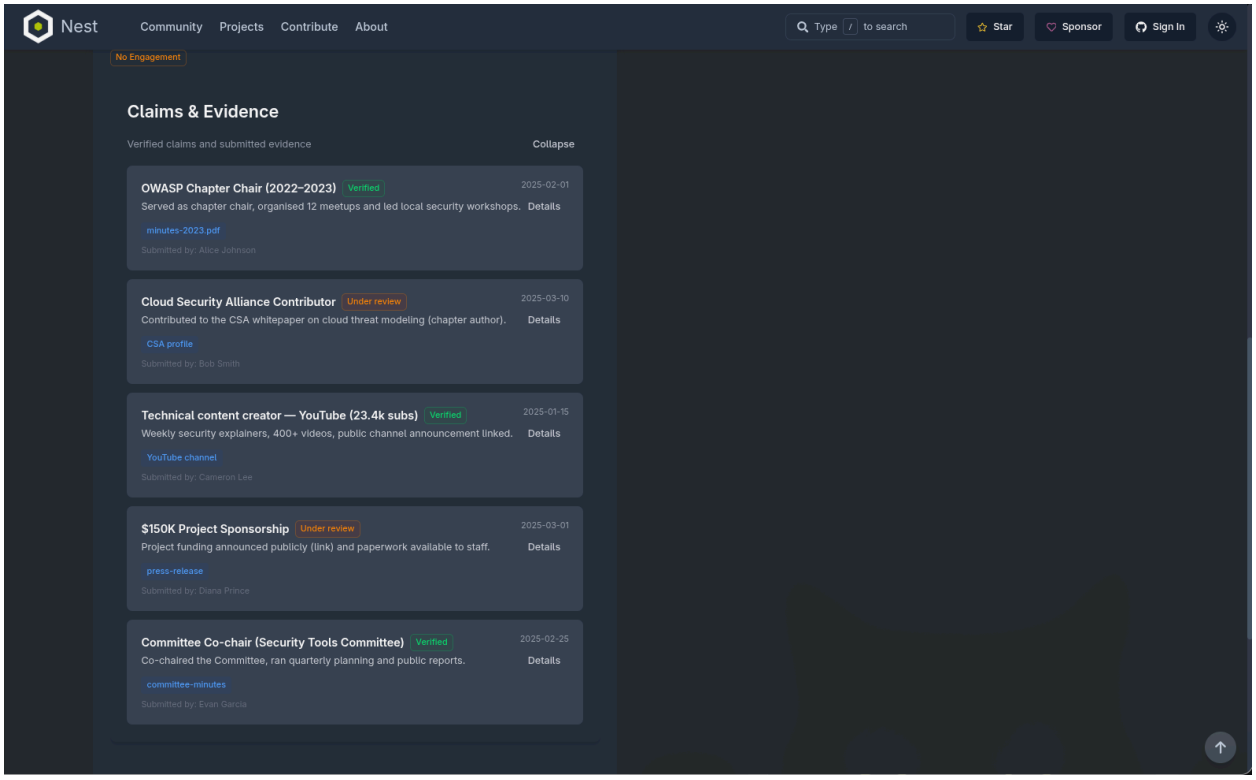

Figure 4 *Prototype design for claims and evidence section in BoD dashboard*

Deliverables

1. A dashboard for the board candidate to submit claims and view their status.
2. A dashboard for community volunteers and OWASP staff to review and annotate candidates' claims.
3. Django models to store immutable candidate claim and review data.
4. An AI-powered candidate page parser to auto generate actionable claims based on candidates' pages.
5. GraphQL queries and mutations to manage claims and reviews.
6. Improvement of existing Board of Directors dashboard by addition of verified candidate claims, activity metrics and sort/filter options.

Risks and Mitigation

1. Risk: Claim evidence verification is ultimately manual and subjective.
   Mitigation: Require multiple reviews for a single claim.
2. Risk: Sensitive claim evidence being accessible to unauthorized users if GraphQL resolver authorization is misconfigured.
   Mitigation: Role-based access control and unit tests.
3. Risk: LLM Parsing is non-deterministic and may produce errors:
   Mitigation: AI generated claims are saved as `DRAFT` by default and require manual verification by the candidate, the candidate may also choose not to use the claim.

Testing and Validation

- ➢ Unit tests: Unit tests will cover single components and functions, I will use Jest and Pytest for this.
- ➢ E2E tests: E2E tests will cover entire pages and API/GraphQL endpoints, I will use Playwright and newly implemented fuzz testing using schemathesis library.

Timeline

Week(s)	**Deliverable(s)**
Week 1	D1, D3, D5: Add Django models to store candidate claim data. Implement GraphQL queries and mutations to submit and view claims.
Week 2	D1, D3, D5: Design a dashboard for the candidate to submit claims including file attachment support. Write tests.
Week 3	D2, D3, D5: Add Django models to store claim review data. Implement GraphQL queries and mutations to submit and view reviews.
Week 4	D2, D3, D5: Design a dashboard for OWASP Staff and community volunteers to review claims. Write tests.
Week 5	D4: Implement an AI-powered parser for backfilling/extracting candidate claim data based on candidate profiles.
Week 6	D6: Add verified candidate claims, activity metrics like GitHub comments, and sort/filter options.

OWASP Board Activity Standardization and Data Programmatic Access

Project Abstract and Goals

The OWASP Board Activity Standardization and Data Programmatic Access project aims to standardize how board activity data is stored and accessed. To achieve this, I will replace the unstructured Markdown-based data in the www-board with a fully standardized, schema-validated, and programmatically accessible system covering meetings, motions, votes, resolutions, discussions, and outcomes. Once standardized, this data will be made available to the community via REST API, GraphQL and auto-generated SDKs.

1. Implement Django models for all types of board actions (discussion, meeting, motion, vote, resolution and outcome).
2. Implement AI-powered parsers to extract existing data from `www-board` repository and store it in the database.
3. Add a sync cron job to sync newly added board actions.
4. Enable programmatic access of the data via REST API, GraphQL, and SDK.

Related Work and Current State

The www-board GitHub repository stores all the board activity related data including meeting discussions, attendance, votes, motions and outcomes. The format for this data is unstructured and mostly stored as Markdown files.

The repository file tree is as follows (only relevant paths are included):

```
├── _data - Contains structured data to be extended
│   ├── attendance.yml
│   ├── board-history.yml
│   ├── members.yml
│   └── votes.yml
├── attendance - Contains only one markdown file
│   │   └── 2020.md
├── elections - Board Election data for each year as markdown files
│   ├── 2012_elections.md
│   ├── ...
│   ├── 2025_elections.md
├── meetings - Meetings that haven't happened yet
│   ├── 202603.md
│   ├── ...
│   ├── 202612.md
```

```
├── meetings-chats - Meeting chats but only for Jan-May, 2020 as text files.
│   ├── 202001.txt
│   ├── ...
│   └── 202005.txt
├── meetings-historical - Meeting data for 2019-2026 as markdown files.
│   ├── 2019
│   ├── ...
│   └── 2026
│       ├── 202601-01.md
│       ├── ...
│       └── 202602.md
├── minutes-deprecated - Meeting Minutes data as markdown files.
│   ├── 2010
│   ├── ...
│   └── 2022
│       ├── 202202.md
│       ├── ...
│       └── 202207.md
```

Detailed Explanation:

_data/

This is the only directory currently using fully structured `.yml` format files.

- `votes.yml`: A historical record of board votes, motions, and their results.
- `attendance.yml`: Yearly records of who attended which board meeting.
- `board-history.yml`: Historical data about board seats, terms, and officers.
- `members.yml`: Information of board members.

meetings/

This directory holds the actively updated markdown files for the current calendar year's board meetings.

- Before a meeting, a file here acts as the **Agenda**.
- After the meeting, the same file is updated with decisions, becoming the **Minutes**.
- Files are typically named by date (e.g., `202601-28.md`).
- Contains a `_template.meeting.md` used as the boilerplate for new meetings.

meetings-historical/

The files from `meetings/` are moved here and organized by year directories (e.g., `2019/`, `2024/`).

- ➢ These files often rely on `_data/votes.yml` to display voting results via **Jekyll**, rather than having the votes hardcoded in the text.
- ➢ The `minutes-deprecated` directory has data hardcoded as text.

minutes-deprecated/
This directory contains older historical meeting records.

- ➢ Meeting files here have data hardcoded inside the markdown itself in the form of a table.
- ➢ This data includes votes, tallies, and motions.

meetings-chats/
Chat logs for some months of the year `2020` only.

attendance/ - Contains a single file for the year 2020.

Benefits to the Community

Board activity data in the `www-board` repository is currently only accessible by reading raw markdown files, making it impractical for any programmatic use.

This project makes the full historical record of OWASP board activities like meetings, motions, votes, and resolutions available via REST API and GraphQL. This enables community members, researchers, and tool builders to query and analyze board activity without manual parsing.

This increases organization transparency and provides the community with queryable evidence of how the board has operated over time.

Approach and Implementation Plan

1. Django Model Design

I will create the following Django models:

- ➢ `BoardMeeting`
- ➢ `BoardMeetingAction`
- ➢ `BoardDiscussion`
- ➢ `BoardMotion`
- ➢ `BoardOutcome`
- ➢ `BoardResolution`
- ➢ `BoardVote`

Based on my research of the `www-board` repository, I have drafted some fields to be added to the Django models for each entity. This requires discussions with mentors to finalize the fields. More fields can also be added later.

`BoardMeeting`:

- `id`: Unique PK
- `board`: FK -> `BoardOfDirectors`
- `attachments`: JSON
- `board_members_absent`: M2M FK -> `EntityMember`
- `board_members_absent_raw`: JSON
- `board_members_present`: M2M FK -> `EntityMember`
- `board_members_present_raw`: JSON
- `call_in_url`: Nullable Text
- `date`: Date
- `guests`: M2M FK -> `EntityMember`
- `guests_raw`: JSON
- `location`: Nullable Text
- `meeting_url`: Nullable Text
- `metadata`: JSON
- `quorum_present`: Nullable Boolean
- `recording_url`: Nullable Text
- `time`: Nullable Text
- `timezone`: Nullable Text
- `title`: Nullable Text
- `type`: Enum(`E_VOTE_ONLY`, `GENERAL`, `PRIVATE`, `PUBLIC`, `SPECIAL` , `SUMMIT`)
- Timestamps (created_at, updated_at) using `TimestampedModel` model

`BoardMeetingAction`:

- `id`: Unique PK
- `order`: Positive Integer
- `meeting`: FK -> `BoardMeeting`
- `discussion`: Nullable FK -> `BoardDiscussion`
- `motion`: Nullable FK -> `BoardMotion`
- `outcome`: Nullable FK -> `BoardOutcome`
- `resolution`: Nullable FK -> `BoardResolution`
- `vote`: Nullable FK -> `BoardVote`
- Timestamps (created_at, updated_at) using `TimestampedModel` model

Constraints:

- unique (meeting, order)

- At least one of `discussion`, `motion`, `outcome`, `resolution`, `vote` must be non-null.
- At most one of `discussion`, `motion`, `outcome`, `resolution`, `vote` must be non-null.

`BoardDiscussion:`

- `id:` Unique PK
- `date`: Nullable Text
- `description`: Text
- `metadata`: JSON
- `participants`: M2M FK -> `EntityMember`
- `participants_raw`: JSON
- `topic`: Text
- Timestamps (created_at, updated_at) using `TimestampedModel` model

`BoardMotion`:

- `id`: Unique PK
- `amends_motion_id`: Nullable FK -> `BoardMotion`
- `background`: Nullable Text
- `date`: Nullable Text
- `description`: Text
- `metadata`: JSON
- `references`: JSON
- `second`: Nullable FK -> `EntityMember`
- `second_raw`: Nullable Text
- `sponsor`: Nullable FK -> `EntityMember`
- `sponsor_raw`: Nullable Text
- `title`: Text
- Timestamps (created_at, updated_at) using `TimestampedModel` model

`BoardOutcome`:

- `id`: Unique PK
- `assignees`: M2M FK -> EntityMember
- `assignees_raw`: JSON
- `description`: Text
- `discussion`: Nullable FK -> BoardDiscussion
- `due_date`: Nullable Date
- `metadata`: JSON
- `status`: Enum(`CANCELLED`, `COMPLETED`, `IN_PROGRESS`, `PENDING`) default pending
- Timestamps (created_at, updated_at) using `TimestampedModel` model

`BoardResolution`:

- ➢ `id`: Unique PK
- ➢ `metadata`: JSON
- ➢ `references`: JSON
- ➢ `text`: Text
- ➢ `vote`: Nullable FK -> `BoardVote`
- ➢ Timestamps (created_at, updated_at) using `TimestampedModel` model

`BoardVote`:

- ➢ `id`: Unique PK
- ➢ `abstain`: M2M FK -> `EntityMember`
- ➢ `abstain_raw`: JSON
- ➢ `against`: M2M FK -> `EntityMember`
- ➢ `against_raw`: JSON
- ➢ `in_favor`: M2M FK -> EntityMember
- ➢ `in_favor_raw`: JSON
- ➢ `metadata`: JSON
- ➢ `motion`: Nullable FK -> BoardMotion
- ➢ `recused`: M2M FK -> EntityMember
- ➢ `recused_raw`: JSON
- ➢ `result`: Enum(`DEFERRED`, `FAILED`, `PASSED`, `TABLED`, `WITHDRAWN`)
- ➢ `tally`: Nullable Text
- ➢ `type`: Enum(`E_VOTE`, `VOTE`)
- ➢ Timestamps (created_at, updated_at) using `TimestampedModel` model

There is a `metadata` field (JSON) in all schemas to account for additional unstructured metadata.
Some fields like `tally` are kept loose as I found HTML tags and inconsistent text while analyzing the repository.

2. AI-Powered Parsing

After the initial models are finalized. I will create Django management commands to parse the existing unstructured data in the `www-board` repository and store it in the database using an external LLM API.

The command will:

- ➢ Fetch markdown files from the www-board GitHub repository using the GitHub API
- ➢ Pass the markdown content to an external LLM API to extract structured meeting data (attendees, motions, votes, discussions, outcomes, resolutions)
- ➢ Store the extracted data into the corresponding Django models

During the process of parsing/storing data, if we encounter fields that are missing or need to be removed, I will update the models right away.

I will also add a cron job that can be configured to run weekly/biweekly/monthly. This job will fetch newly added meeting minutes from the `www-board` repository and run them through the same parsing pipeline, this will keep the Nest database synced without manual intervention.

3. Post-Population Checks and Actions

Once the new migrated data is stored into the database, we will manually verify and edit the migrated data to ensure correctness.

4. Data Programmatic Access

I will extend the Nest REST API with endpoints to make the board activity data available to the community for programmatic use. I will also extend the GraphQL nodes with this data. The SDKs are automatically generated using Speakeasy.

Meetings:
Endpoints: `/api/v0/board/meetings/` and `/api/v0/board/meetings/{meeting_id}`

- ➢ List: filter by type (enum: (`E_VOTE_ONLY`, `GENERAL`, `PRIVATE`, `PUBLIC`, `SPECIAL` , `SUMMIT`), `date_gte`, `date_lte`, `quorum_present` (boolean), ordering by date.
- ➢ Detail: full meeting object including `actions`, `board_members_present`, `board_members_absent`, `guests`, `recording_url`, `call_in_url`.

Motions:
Endpoints: `/api/v0/board/motions/` and `/api/v0/board/motions/{motion_id}`

- ➢ List: filter by `sponsor`, `date_gte`, `date_lte`, ordering by date.
- ➢ Detail: full motion object including `sponsor`, `second`, `description`, `amends_motion_id`, `references`.

Votes:
Endpoints: `/api/v0/board/votes/` and `/api/v0/board/votes/{vote_id}`

- ➢ List: filter by result (enum: `DEFERRED`, `FAILED`, `PASSED`, `TABLED`, `WITHDRAWN`), type (enum: `E_VOTE`, `VOTE`), `motion_id`, ordering by date.
- ➢ Detail: full vote object including `in_favor`, `against`, `abstain`, `recused`, `tally`, `motion_id`.

Deliverables

1. Django models (`BoardMeeting`, `BoardMeetingAction`, `BoardMotion`, `BoardDiscussion`, `BoardOutcome`, `BoardResolution`, `BoardVote`).
2. Django management commands to parse historical data and populate the database.

3. A cron sync job to sync newly added board actions.
4. REST API endpoints and GraphQL nodes exposing board activity data.

Risks and Mitigation

1. Risk: Using AI to parse markdown data might produce silent incorrect records in the database.
   Mitigation: Verify with a smaller subset of data, manual verification of parsed data.

Testing and Validation

- ➢ Unit Tests: I will use Pytest to test the management commands.
- ➢ Manual Verification: After the initial database population, I will manually verify a sample of parsed records against the original markdown files to ensure correctness.

Timeline

Week(s)	**Deliverable(s)**
Week 7	D1: Finalize and implement all six Django models with mentor input.
Week 8	D2: Implement Django management command to parse and populate the database.
Week 9	D2, D3: Continue working on the management commands and write tests.
Week 10	D4: Implement REST API endpoints and GraphQL nodes/queries.
Week 11	D4: Continue working on REST API endpoints and GraphQL and add tests.
Week 12	Buffer week to account for unexpected delays.

LLM Usage Considerations

LLMs were used for supportive or editorial purposes in this work. All outputs were reviewed and validated by the contributor to ensure accuracy and originality. Usage includes formatting text and improving grammar.

Collaboration and Availability

Communication medium:
The following platforms/channels will be used for communication and collaboration:

- The OWASP Slack workspace

- OWASP Nest weekly community huddles
- OWASP Nest LinkedIn group

Availability:
I do not have any other commitments.

How many proposals have I submitted?
I have submitted only one proposal for GSoC'26 (this one).